



Roadmap AI Engineering 2026



Perfil: @matheusolivsilv

Foco: **AI Engineering (product)** construir produto com modelo pronto, não treinar modelo do zero.

Como usar este roadmap

1. Siga a **ordem**. Pular etapa é o erro mais comum.
2. Cada bloco tem **o que estudar + link pra começar**.
3. Projeto do zero (chatbot → RAG → produção) fica pra outro conteúdo — aqui é o mapa.

Passo 0 — Linguagem

Python é o padrão em vaga de AI Engineering.

Se você é...	Comece por
Iniciante em programação	Learn Python (interativo)
Já programa em outra linguagem	Python Tutorial oficial
Dev web / full stack com IA	TypeScript Handbook como segunda linguagem
Mirando treinamento de modelo	Python + no futuro C++ (não é prioridade agora)

Meta: conseguir ler código Python, instalar pacote com `pip`, rodar script local.

Passo 1 — Escolha o caminho

O roadmap muda dependendo do que você quer:

Caminho	O que você faz no dia a dia
AI Engineering ← este roadmap	API, RAG, agentes, contexto, deploy de produto com LLM
ML Engineer	Treinar/fine-tunar modelo, dados, experimentos
Full stack com IA	App + back-end + integração com modelo

Este doc é **AI Engineering**. Se escolheu outro, alguns blocos ainda servem (Python, fundamentos), mas a ordem muda.

Passo 2 — Fundamentos de LLM (alto nível)

Sem virar matemático. Você precisa **intuição**, não prova.

O que dominar

- **Tokens** — por que você paga, trunca e quebra por token
- **Embeddings** — texto vira vetor; similaridade semântica
- **Transformer (básico)** — atenção, autoregressão: o modelo prevê o próximo token
- **Prompt** — consequência dos fundamentos, não “prompt engineering” solto

Onde estudar

Recurso	Link
LLM para devs (curto, gratuito)	DeepLearning.AI — Generative AI for Everyone
NLP / embeddings (gratuito)	Hugging Face — NLP Course
Mapa visual da área	roadmap.sh — AI Engineer
Livro (referência de longo prazo)	AI Engineering — Chip Huyen

Meta: explicar em 2 minutos como palavra vira número e como o modelo gera texto.

Passo 3 — API generativa na prática

Antes de framework, chama a API **na mão** uma vez.

O que dominar

- Request com e sem **streaming** (ver token chegando)
- Estrutura da resposta (mensagem human vs assistant)
- **Tool calling** — o que significa o modelo “chamar uma tool”
- Output estruturado (JSON / schema)
- Áudio e imagem (só saber que existem — não precisa dominar agora)

Onde estudar

Recurso	Link
Quickstart OpenAI	platform.openai.com/docs/quickstart
Streaming	OpenAI — Streaming
Function / tool calling	OpenAI — Function calling
Anthropic (alternativa)	docs.anthropic.com — Get started

Meta: script local que streama resposta e um que chama uma tool fake (ex: `get_weather`).

Passo 4 — LangChain + LangGraph



Este é o passo que não cabe no reel — e o que separa quem só “usou ChatGPT” de quem constrói produto.

Depois de entender API crua, use um framework pra não reinventar roda: trocar provider, tools, memória, streaming no front.

O que dominar

- Trocar provider (GPT ↔ Claude ↔ Gemini) com o mesmo código
- Contrato: input → output → tool calls → tool results
- **LangGraph** — grafo de estados, agente ReAct simples
- Streaming events e impacto no UX da UI

- Por que framework primeiro, internals depois

Onde estudar (gratuito)

Recurso	Link
LangChain Academy (comece aqui)	academy.langchain.com
Intro LangChain + agentes	Introduction to LangChain — Python
Intro LangGraph	Introduction to LangGraph — Python
Notebooks oficiais (clone local)	github.com/langchain-ai/langchain-academy
Docs LangChain	docs.langchain.com
Docs LangGraph	docs.langchain.com/oss/python/langgraph/overview

Ordem sugerida: Academy “Introduction to LangChain” → “Introduction to LangGraph” → notebooks no GitHub.

Meta: chatbot com 1 tool funcionando local + entender o grafo do agente.

Passo 5 — Contexto, RAG e padrões de produto

O que dominar

- **Janela de contexto** — o que cabe, o que estoura, compaction
- **RAG (alto nível)** — ingestão vs retrieval; chunk; quando RAG vs prompt puro
- **Memória** — curto prazo (thread) vs longo prazo (perfil do usuário)
- **Padrões que aparecem em vaga e entrevista:** MCP, Skills, multi-agent

Onde estudar

Tópico	Link
RAG conceitual (curto)	DeepLearning.AI — LangChain for LLM Application Development
MCP — o que é	modelcontextprotocol.io
Vector DB (conceito)	pgvector — GitHub

Meta: explicar como você entregaria um PDF de 80 páginas pro modelo sem mandar tudo no prompt.

Passo 6 — A ordem que a maioria erra (a 4ª coisa do vídeo)

Python → Fundamentos LLM → API na mão → LangChain/LangGraph → Contexto/RAG → Produção

Erro comum	Por que trava
Pular direto pra “fazer um agente”	Não entende token, tool call nem contexto
Só tutorial de prompt	Não sabe o que acontece quando estoura contexto
Framework sem API crua	Não debuga quando quebra em prod
RAG antes de embedding	Chunk e score viram mágica negra

Software engineering clássico (banco, auth, observabilidade, deploy) você provavelmente já tem — é o mesmo back-end de sempre. O diferencial é dominar passos 2–5.

Próximo passo

Curte nosso post pra dar aquela moral, e segue o perfil!

Projeto guiado do zero (chatbot → documentos → retrieval → eval) — acompanha o perfil [@matheusolivsilv](#).

Sobre

Conteúdo do [@matheusolivsilv](#) · AI Engineering em produção.

julho/2026